MODUL PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 1

React Native dengan Expo

Instalasi Environment, Pengenalan Komponen, Props, State & Events

Mata Kuliah **Pemrograman Mobile**	Framework **React Native + Expo SDK**
Emulator / Device **Android Studio AVD (Android Virtual Device)**	Semester / SKS **Genap / 3 SKS (1 Teori + 2 Praktikum)**

Identitas Mahasiswa

Nama Lengkap	:	
NIM	:	
Kelas / Kelompok	:	
Tanggal Praktikum	:	
Nama Asisten / Dosen	:	
Nilai	:	

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Membedakan React Native dengan React Web dan memahami mengapa React Native digunakan untuk mobile
2. Menginstal dan mengkonfigurasi environment React Native dengan Expo CLI
3. Membuat dan menjalankan Android Virtual Device (AVD) di Android Studio
4. Menghubungkan project Expo ke emulator Android Studio
5. Memahami komponen dasar React Native: View, Text, StyleSheet, TouchableOpacity, TextInput
6. Membuat komponen fungsional React Native yang reusable dengan props
7. Mengelola state komponen menggunakan Hook useState()
8. Menangani event touch dan input pada komponen React Native

Alat dan Bahan

Perangkat & Software

Laptop/PC RAM minimal 16 GB (emulator berat!)

Prosesor 64-bit dengan virtualisasi (Intel VT-x / AMD-V)

Storage minimal 10 GB tersedia (Android Studio + AVD)

Node.js v18 LTS atau lebih baru

Android Studio (versi terbaru — Hedgehog/Koala)

VS Code dengan ekstensi React Native Tools

Git (disarankan)

Yang Akan Diinstal

Android SDK (diinstal via Android Studio)

Android Virtual Device / AVD (Pixel 6, API 33+)

Expo CLI: npm install -g expo-cli

EAS CLI (opsional, untuk build APK nanti)

Ekstensi VS Code: React Native Tools (Microsoft)

Ekstensi VS Code: ES7+ React Snippets

Expo Go (di HP fisik, opsional backup)

Perhatian Spesifikasi Minimum

Android Emulator memerlukan hardware virtualization yang aktif di BIOS.
Jika laptop Anda memiliki RAM < 16 GB atau tidak mendukung virtualisasi,
gunakan HP fisik dengan aplikasi Expo Go sebagai alternatif.
Cara cek: buka Task Manager > Performance > CPU > lihat 'Virtualization: Enabled'

Materi Praktikum

Bagian 1: React Native vs React Web

1.1 Perbedaan Mendasar

React Native dan React Web menggunakan konsep yang sama (component, props, state, hooks), namun memiliki perbedaan fundamental pada layer rendering-nya.

Aspek	React Web	React Native
Output	Website (HTML/CSS/JS)	**Aplikasi Android/iOS native**
Komponen UI	, <p>, <button>	**<View>, <Text>, <TouchableOpacity>**
Styling	CSS / className	**StyleSheet.create() — mirip CSS**
Navigasi	React Router / URL	**React Navigation (stack/tab/drawer)**
Akses Device	Terbatas (browser sandbox)	**Penuh: kamera, GPS, sensor, dll.**
Build Tool	Vite / Webpack	**Metro Bundler (via Expo)**
Testing	Browser	**Emulator Android/iOS atau HP fisik**

1.2 Komponen Dasar React Native

Berikut adalah komponen-komponen inti React Native yang paling sering digunakan:

Komponen	Fungsi & Padanan HTML
<View>	Container utama — setara . Digunakan untuk layout dan grouping elemen.
<Text>	Menampilkan teks — setara <p>, <h1>, . Semua teks WAJIB di dalam <Text>.
<StyleSheet>	API untuk mendefinisikan style — mirip CSS, tapi dalam objek JavaScript.
<TouchableOpacity>	Tombol yang bisa ditekan — setara <button>. Memberikan efek opacity saat ditekan.
<TextInput>	Input teks dari user — setara <input type='text'>.
<ScrollView>	Konten yang bisa di-scroll — setara overflow: scroll pada div.
<FlatList>	Render daftar panjang secara efisien — pengganti map() untuk list besar.
<Image>	Menampilkan gambar lokal atau dari URL — setara <img>.
<Alert>	Dialog/popup native — mirip window.alert() tapi tampil sebagai dialog sistem.

Bagian 2: Instalasi Environment

2.1 Instalasi Node.js

9. Kunjungi https://nodejs.org dan download versi LTS
10. Jalankan installer — ikuti langkah default
11. Verifikasi instalasi di Terminal / Command Prompt:

```
node --version    # Output: v20.x.x
npm --version     # Output: 10.x.x
```

2.2 Instalasi Android Studio

Android Studio diperlukan untuk menyediakan Android SDK dan Android Virtual Device (AVD) yang akan digunakan sebagai emulator.

1	**Download Android Studio** Kunjungi https://developer.android.com/studio dan download Android Studio versi terbaru untuk sistem operasi Anda (Windows/Mac/Linux).
2	**Jalankan Installer** Buka installer yang sudah didownload. Saat proses setup, pastikan komponen berikut dicentang: **Catatan** Android SDK | Android SDK Platform | Android Virtual Device | Performance (Intel HAXM untuk Intel, atau Hyper-V untuk AMD)
3	**Selesaikan Setup Wizard** Setelah instalasi, Android Studio akan membuka Setup Wizard. Pilih 'Standard' untuk konfigurasi default. Android Studio akan mengunduh SDK yang diperlukan (membutuhkan koneksi internet, ~2-3 GB).
4	**Konfigurasi Android SDK** Buka Android Studio > More Actions > SDK Manager. Pastikan tab 'SDK Platforms' memiliki Android 13.0 (API Level 33) atau yang lebih baru tercentang. Di tab 'SDK Tools', centang: Android SDK Build-Tools, Android Emulator, Android SDK Platform-Tools.

2.3 Konfigurasi Environment Variables

Android SDK harus didaftarkan sebagai environment variable agar dapat diakses dari Terminal.

Windows — Langkah Konfigurasi

1. Buka System Properties > Advanced > Environment Variables
2. Di bagian 'System Variables', klik New:
   Variable Name : ANDROID_HOME
   Variable Value: C:\Users\[NamaUser]\AppData\Local\Android\Sdk
3. Cari variabel 'Path', klik Edit > New, tambahkan dua baris berikut:
   %ANDROID_HOME%\platform-tools

%ANDROID_HOME%\emulator

4. Klik OK dan restart Terminal/Command Prompt

macOS / Linux — Langkah Konfigurasi

Tambahkan baris berikut ke file ~/.bashrc atau ~/.zshrc:

export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$PATH:$ANDROID_HOME/emulator
export PATH=$PATH:$ANDROID_HOME/platform-tools

Kemudian jalankan: source ~/.bashrc (atau source ~/.zshrc)

Verifikasi konfigurasi berhasil dengan menjalankan perintah berikut:

```
adb --version
# Output yang beharapkan: Android Debug Bridge version 1.x.x
```

2.4 Membuat Android Virtual Device (AVD)

AVD adalah emulator Android yang berjalan di komputer Anda. Kita akan membuat AVD untuk simulasi smartphone Android.

1 Buka AVD Manager

Di halaman utama Android Studio, klik 'More Actions' (ikon tiga titik) lalu pilih 'Virtual Device Manager'. Atau dari dalam project, buka menu Tools > Device Manager.

2 Buat Device Baru

Klik tombol '+' (Create virtual device). Pilih kategori 'Phone', kemudian pilih device 'Pixel 6'. Klik Next.

3 Pilih System Image

Pilih tab 'Recommended'. Download dan pilih 'Tiramisu' (API Level 33, Android 13) atau yang lebih baru. Klik tombol Download di sebelah nama image jika belum tersedia, tunggu hingga selesai. Setelah selesai, klik Next.

4 Konfigurasi AVD

Beri nama AVD, misalnya 'Pixel_6_API33'. Pada bagian Startup orientation, pilih Portrait. Klik Finish.

> **Catatan**
> Untuk performa lebih baik: klik 'Show Advanced Settings' dan atur RAM ke 2048 MB atau lebih jika RAM laptop Anda mencukupi.

5 Jalankan Emulator

Pada daftar AVD, klik tombol Play (segitiga) di sebelah kanan AVD yang baru dibuat. Tunggu hingga emulator menyala — bisa memakan waktu 2-5 menit pertama kali.

> **Catatan**
> Biarkan emulator tetap menyala selama sesi praktikum untuk menghemat waktu.

2.5 Instalasi Expo CLI dan Membuat Project

Expo adalah platform yang mempermudah development React Native. Dengan Expo, kita tidak perlu mengkonfigurasi Android SDK secara manual untuk development.

```
# Install Expo CLI secara global
npm install -g expo-cli

# Verifikasi instalasi
expo --version
```

Buat project React Native baru menggunakan Expo:

```
# Buat project baru
npx create-expo-app praktikum-rn-1 -template blank

# Pilih template: Blank (JavaScript)
```

```
# Masuk ke folder project
cd praktikum-rn-1
```

Struktur Folder Project Expo

app.json -> Konfigurasi aplikasi (nama, ikon, splash screen)
App.js -> Komponen utama — titik awal aplikasi
assets/ -> Gambar, font, dan aset lainnya
node_modules/ -> Library yang terinstal (jangan diedit manual)
package.json -> Daftar dependencies dan script perintah
babel.config.js -> Konfigurasi transpiler JavaScript

2.6 Menghubungkan Expo ke Android Studio AVD

Ini adalah langkah krusial: menjalankan project Expo di emulator Android Studio.

1 Pastikan emulator sudah menyala

Buka Android Studio, pergi ke Device Manager, dan pastikan AVD (Pixel 6 API 33) sudah dalam status Running (ikon hijau). Jika belum, klik tombol Play untuk menjalankannya.

2 Buka project di VS Code

Dari terminal, buka folder project di VS Code:

```
cd praktikum-rn-1
code .
```

3 Jalankan Expo Dev Server

Di terminal VS Code (Ctrl+` untuk membuka), jalankan perintah berikut:

```
npx expo start

# Atau jika ingin langsung ke Android:
npx expo start --android
```

Catatan

Expo akan menampilkan QR code dan beberapa opsi di terminal.

4 Buka di Emulator Android

Setelah server berjalan, tekan huruf 'a' di terminal untuk membuka aplikasi di emulator Android. Expo akan menginstal Expo Go secara otomatis ke emulator (hanya pertama kali), lalu membuka project Anda.

```
# Di terminal yang sedang menjalankan Expo:
# Tekan 'a'  → buka di Android emulator
# Tekan 'r'  → reload aplikasi
# Tekan 'm'  → buka developer menu
# Tekan 'j'  → buka Hermes debugger
# Ctrl+C     → hentikan server
```

5 Verifikasi Berhasil

Jika berhasil, emulator Android akan menampilkan aplikasi default Expo dengan teks 'Open up App.js to start working on your app!' Ini berarti lingkungan development sudah siap digunakan.

Troubleshooting Umum

Error 'adb not found': Environment variable ANDROID_HOME belum dikonfigurasi — ulangi langkah 2.3

Emulator tidak terdeteksi: Pastikan emulator sudah benar-benar menyala sebelum menjalankan expo start

Error ENOSPC (Linux): Jalankan: echo fs.inotify.max_user_watches=524288 | sudo tee -a /etc/sysctl.conf

Expo Go crash: Coba tekan 'a' lagi atau restart emulator lalu tekan 'a'

Layar hitam di emulator: Geser layar emulator ke atas (unlock screen)

Bagian 3: Komponen, Props, State & Events

3.1 Struktur Dasar Komponen React Native

```
// App.js — Struktur dasar komponen React Native
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.judul}>Halo, React Native!</Text>
      <Text style={styles.sub}>Aplikasi mobile pertama saya</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  judul: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#1F4E79',
  },
  sub: {
    fontSize: 14,
    color: '#666',
    marginTop: 8,
  },
});
```

3.2 Props dan Komponen Reusable

```
// components/KartuProfil.js
import { StyleSheet, Text, View } from 'react-native';

export default function KartuProfil({ nama, nim, prodi, angkatan }) {
  return (
    <View style={styles.kartu}>
      <Text style={styles.nama}>{nama}</Text>
      <Text style={styles.info}>NIM    : {nim}</Text>
      <Text style={styles.info}>Prodi  : {prodi}</Text>
      <Text style={styles.info}>Angkatan: {angkatan}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  kartu: {
    borderWidth: 1,
    borderColor: '#2E75B6',
    borderRadius: 12,
    padding: 16,
    margin: 8,
    backgroundColor: '#f0f8ff',
  },
  nama: { fontSize: 18, fontWeight: 'bold', color: '#1F4E79', marginBottom: 6
},
  info: { fontSize: 14, color: '#444', marginBottom: 2 },
});
```

3.3 State dengan useState dan Event Handler

```
// components/Counter.js
import { useState } from 'react';
import { StyleSheet, Text, View, TouchableOpacity, Alert } from 'react-native';

export default function Counter() {
  const [count, setCount] = useState(0);

  const handleTambah = () => setCount(count + 1);
  const handleKurang = () => {
    if (count === 0) {
      Alert.alert('Peringatan', 'Nilai tidak bisa kurang dari 0!');
      return;
    }
    setCount(count - 1);
  };
  const handleReset = () => setCount(0);

  return (
    <View style={styles.container}>
      <Text style={styles.angka}>{count}</Text>
      <View style={styles.baris}>
        <TouchableOpacity style={styles.tombol} onPress={handleKurang}>
          <Text style={styles.tombolTeks}>-</Text>
        </TouchableOpacity>
        <TouchableOpacity style={[styles.tombol, styles.reset]}
onPress={handleReset}>
          <Text style={styles.tombolTeks}>Reset</Text>
        </TouchableOpacity>
        <TouchableOpacity style={styles.tombol} onPress={handleTambah}>
          <Text style={styles.tombolTeks}>+</Text>
        </TouchableOpacity>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { alignItems: 'center', padding: 20 },
  angka: { fontSize: 64, fontWeight: 'bold', color: '#1F4E79', marginBottom: 20
},
  baris: { flexDirection: 'row', gap: 12 },
  tombol: { backgroundColor: '#2E75B6', borderRadius: 8, padding: 14, minWidth:
60, alignItems: 'center' },
  reset: { backgroundColor: '#888' },
  tombolTeks: { color: '#fff', fontSize: 20, fontWeight: 'bold' },
});
```

Perbedaan Penting: Web vs React Native

Web : onClick -> React Native: onPress
Web : <button> -> React Native: <TouchableOpacity>
Web : alert() -> React Native: Alert.alert('Judul', 'Pesan')
Web : CSS class -> React Native: StyleSheet.create({ }) + objek style
Web : flexbox default row -> React Native: flexbox default COLUMN

Langkah-Langkah Praktikum

Praktikum A — Setup Environment & Verifikasi

Target

Project Expo berjalan di emulator Android Studio AVD. Perkiraan waktu: 30-45 menit

1 Verifikasi semua tools sudah terinstal

Buka Terminal dan jalankan perintah berikut satu per satu untuk memastikan semua tools siap:

```
node --version      # Harus v18 ke atas
npm --version       # Harus v9 ke atas
adb --version       # Harus muncul versi ADB
expo --version      # Harus muncul versi Expo CLI
```

2 Nyalakan emulator Android

Buka Android Studio > Device Manager > klik tombol Play di sebelah AVD Pixel 6 API 33. Tunggu hingga layar Android muncul dan emulator siap digunakan.

Catatan

Jangan tutup Android Studio atau emulator selama praktikum berlangsung!

3 Buat project Expo baru

Buka Terminal, navigasi ke folder kerja, lalu buat project baru:

```
cd Desktop
npx create-expo-app praktikum-a1
# Pilih template: Blank
cd praktikum-a1
code .
```

4 Jalankan Expo dan hubungkan ke emulator

Di terminal VS Code, jalankan Expo lalu tekan 'a' untuk membuka di emulator Android:

```
npx expo start
# Setelah terminal menampilkan menu, tekan tombol 'a'
```

Catatan

Proses pertama kali lebih lama karena Expo perlu menginstal Expo Go ke emulator. Tunggu hingga aplikasi muncul di emulator.

5 Uji Hot Reload

Buka file App.js di VS Code, ubah teks 'Open up App.js...' menjadi 'Halo! Ini Aplikasi Pertama Saya — [Nama Anda]'. Simpan file (Ctrl+S). Amati perubahan otomatis di emulator tanpa perlu restart.

Catatan

Fitur ini disebut Fast Refresh — setiap perubahan kode langsung terlihat di emulator dalam hitungan detik.

Praktikum B — Membuat Komponen Kartu Profil

Target

Membuat komponen KartuProfil yang reusable dan menampilkan minimal 3 kartu menggunakan props. Perkiraan waktu: 25 menit

12. Buat folder components di dalam folder project: src/components/
13. Buat file KartuProfil.js di dalam folder tersebut
14. Ketik kode komponen KartuProfil dari materi 3.2 di atas
15. Update App.js untuk menggunakan ScrollView dan menampilkan beberapa kartu:

```
// App.js
import { StyleSheet, Text, View, ScrollView } from 'react-native';
import KartuProfil from './components/KartuProfil';

export default function App() {
  return (
    <ScrollView style={styles.container}>
      <Text style={styles.judul}>Daftar Mahasiswa</Text>
      <KartuProfil
        nama='[Nama Anda]'
        nim='[NIM Anda]'
        prodi='Teknik Informatika'
        angkatan='2023'
      />
      <KartuProfil
        nama='Teman Sekelas'
        nim='20230002'
        prodi='Teknik Informatika'
        angkatan='2023'
      />
      <KartuProfil
        nama='Mahasiswa Lain'
        nim='20230003'
        prodi='Sistem Informasi'
        angkatan='2022'
      />
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5', paddingTop: 50 },
  judul: { fontSize: 22, fontWeight: 'bold', textAlign: 'center',
           color: '#1F4E79', marginBottom: 12 },
});
```

16. Simpan dan amati hasilnya di emulator — scroll untuk melihat semua kartu
17. Tambahkan minimal satu kartu lagi dengan data yang berbeda
18. Modifikasi style kartu — ubah warna border, font size, atau tambahkan elemen baru

Praktikum C — Aplikasi Counter Interaktif

Target

Membuat Counter dengan useState, onPress, dan Alert. Perkiraan waktu: 25 menit

19. Buat file Counter.js di folder components/
20. Ketik kode Counter dari materi 3.3 di atas
21. Import dan gunakan komponen Counter di App.js
22. Uji semua tombol: Tambah, Kurang, Reset
23. Verifikasi bahwa Alert muncul sebagai dialog native di emulator saat nilai mencoba menjadi negatif
24. Tambahkan fitur berikut secara mandiri:
    - Ubah warna angka menjadi merah jika nilai di atas 10
    - Tambahkan batas maksimum 20 dengan Alert peringatan
    - Tampilkan teks status: 'Rendah' (0-5), 'Sedang' (6-10), 'Tinggi' (>10)

Latihan Mandiri

Kerjakan latihan berikut secara mandiri. Latihan ini dirancang untuk mensimulasikan fitur nyata pada aplikasi mobile.

Latihan 1 — Aplikasi Profil Diri (Mobile Card)

Buat aplikasi yang menampilkan halaman profil diri sendiri layaknya profil di aplikasi mobile. Komponen utama bernama ProfilApp.

Fitur yang wajib ada:

25. Header dengan warna latar biru dan nama lengkap berwarna putih
26. Avatar placeholder (View berbentuk lingkaran dengan inisial nama)
27. Informasi profil: NIM, Program Studi, Semester, Email
28. Minimal 3 tombol aksi: Edit Profil, Bagikan, Hubungi — masing-masing menampilkan Alert berbeda saat ditekan
29. Bagian 'Keahlian' yang menampilkan daftar skill menggunakan komponen Tag yang reusable

Petunjuk Komponen Tag

Buat komponen Tag.js yang menerima props: label (string) dan warna (string).

Tampilkan sebagai kotak kecil berwarna dengan teks di dalamnya.

Contoh penggunaan: <Tag label='JavaScript' warna='#F7DF1E' />
<Tag label='React Native' warna='#61DBFB' />

Latihan 2 — Kalkulator Sederhana Mobile

Buat aplikasi kalkulator sederhana yang berjalan di emulator Android. Layout harus menyerupai kalkulator pada umumnya.

Fitur yang wajib ada:

30. Display angka di bagian atas (menampilkan input dan hasil)
31. Tombol angka 0-9 tersusun dalam grid (gunakan flexDirection: 'row' dan flexWrap: 'wrap')
32. Tombol operasi: + (tambah), - (kurang), x (kali), / (bagi), = (hitung), C (clear)
33. Validasi: tampilkan pesan 'Error: Pembagian dengan nol!' jika dibagi 0
34. Tampilan angka desimal maksimal 2 angka di belakang koma

Spesifikasi Styling

Tombol angka: background abu-abu (#E0E0E0), teks hitam

Tombol operator (+, -, x, /): background biru (#2E75B6), teks putih

Tombol = : background hijau (#388E3C), teks putih, lebar penuh

Tombol C : background merah (#D32F2F), teks putih
Semua tombol: borderRadius: 8, padding minimal 16

Tantangan Tambahan (Nilai Bonus)

Tambahkan riwayat perhitungan yang ditampilkan di ScrollView di bawah display
Tambahkan tombol +/- untuk mengubah tanda positif/negatif
Tambahkan tombol % untuk perhitungan persentase
Buat layout responsif yang terlihat baik di berbagai ukuran layar emulator

Lembar Kerja dan Dokumentasi

Pertanyaan Analitis

Jawab pertanyaan berikut berdasarkan pemahaman dan hasil praktikum Anda:

1. Jelaskan mengapa React Native menggunakan <View> dan <Text> bukan dan <p>! Apa yang terjadi jika Anda mencoba menulis teks langsung di luar komponen <Text> di React Native?

2. Apa perbedaan antara onClick (React Web) dan onPress (React Native)? Mengapa React Native tidak menggunakan onClick?

3. Dalam React Native, StyleSheet menggunakan Flexbox dengan default direction 'column', sedangkan di web CSS default-nya 'row'. Bagaimana hal ini mempengaruhi cara Anda menyusun layout di React Native?

4. Jelaskan apa yang dimaksud dengan 'Hot Reload' / 'Fast Refresh' di Expo! Mengapa fitur ini sangat membantu proses development?

Dokumentasi Screenshot

Tempelkan screenshot hasil praktikum di emulator Android (bukan browser) di bawah ini:

Screenshot 1: Praktikum A — Tampilan awal Expo berhasil di emulator (dengan teks nama Anda)

[Tempelkan screenshot emulator Android di sini]

Screenshot 2: Praktikum B — Halaman scroll berisi minimal 3 KartuProfil

[Tempelkan screenshot emulator Android di sini]

Screenshot 3: Praktikum C — Counter berfungsi (tunjukkan juga Alert peringatan)

[Tempelkan screenshot emulator Android di sini]

Screenshot 4: Latihan 1 — Halaman ProfilApp lengkap dengan tag keahlian

[Tempelkan screenshot emulator Android di sini]

Rubrik Penilaian

Komponen	Bobot	Kriteria Penilaian
Praktikum A — Setup & Emulator	20%	Expo berhasil terhubung ke emulator AVD, fast refresh berfungsi
Praktikum B — Komponen & Props	20%	KartuProfil reusable, minimal 3 kartu tampil dengan data berbeda
Praktikum C — State & Events	20%	Counter berfungsi, Alert muncul, fitur tambahan dikerjakan
Latihan Mandiri	25%	Minimal satu latihan selesai dan berfungsi di emulator
Pertanyaan Analitis	10%	Jawaban menunjukkan pemahaman konsep, bukan copy-paste
Screenshot Dokumentasi	5%	Screenshot dari emulator Android (bukan browser)

Catatan Asisten / Dosen

Nilai Akhir Praktikum:

Tanda Tangan Asisten / Dosen